



TACTICAL PUZZLE ENGINE

محمد عادل الكيلاني

رجب خالد الكبتي

عبدالبارئ السنوسي الشامخ



نبذة عامة عن

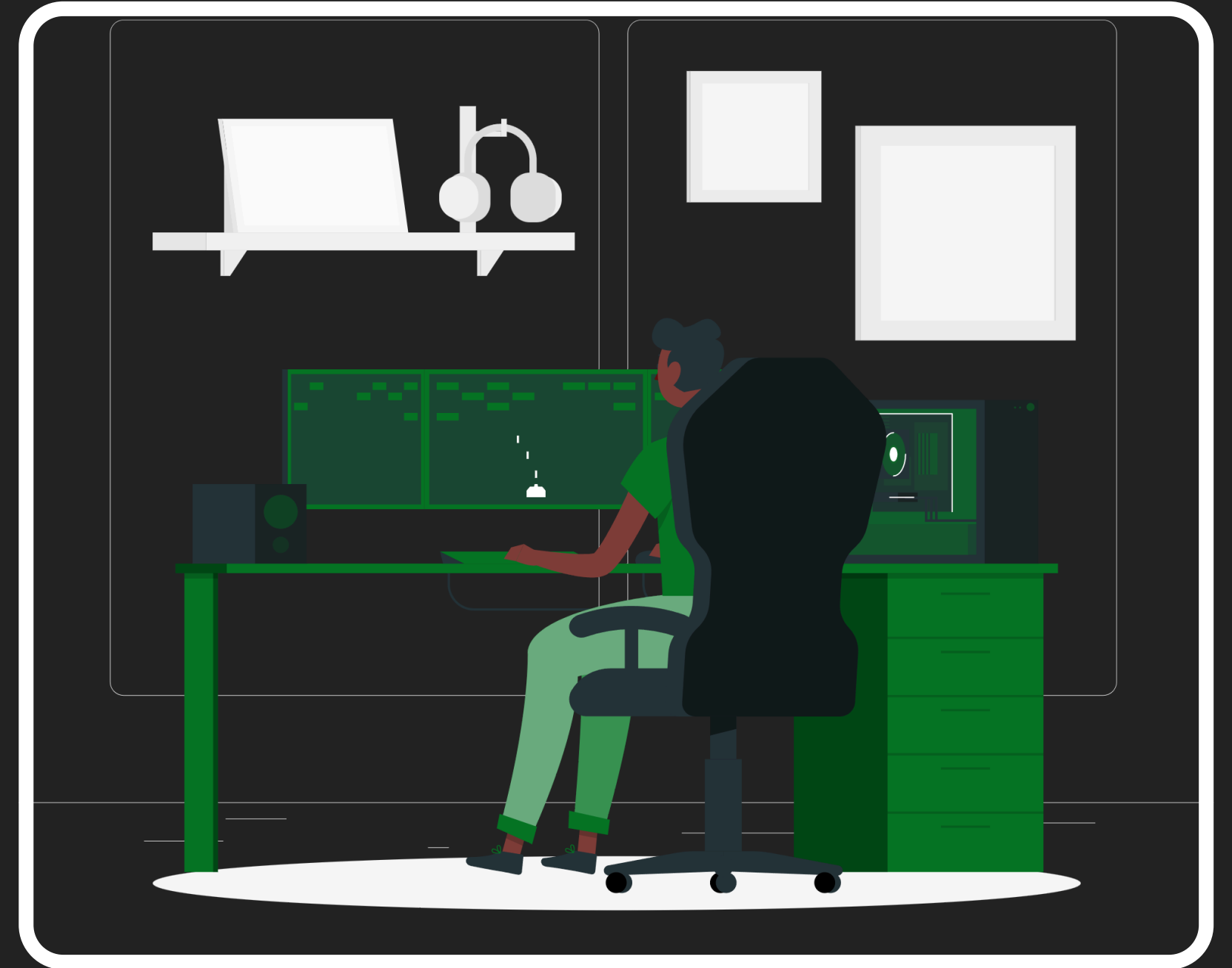
Tactical Puzzle Engine

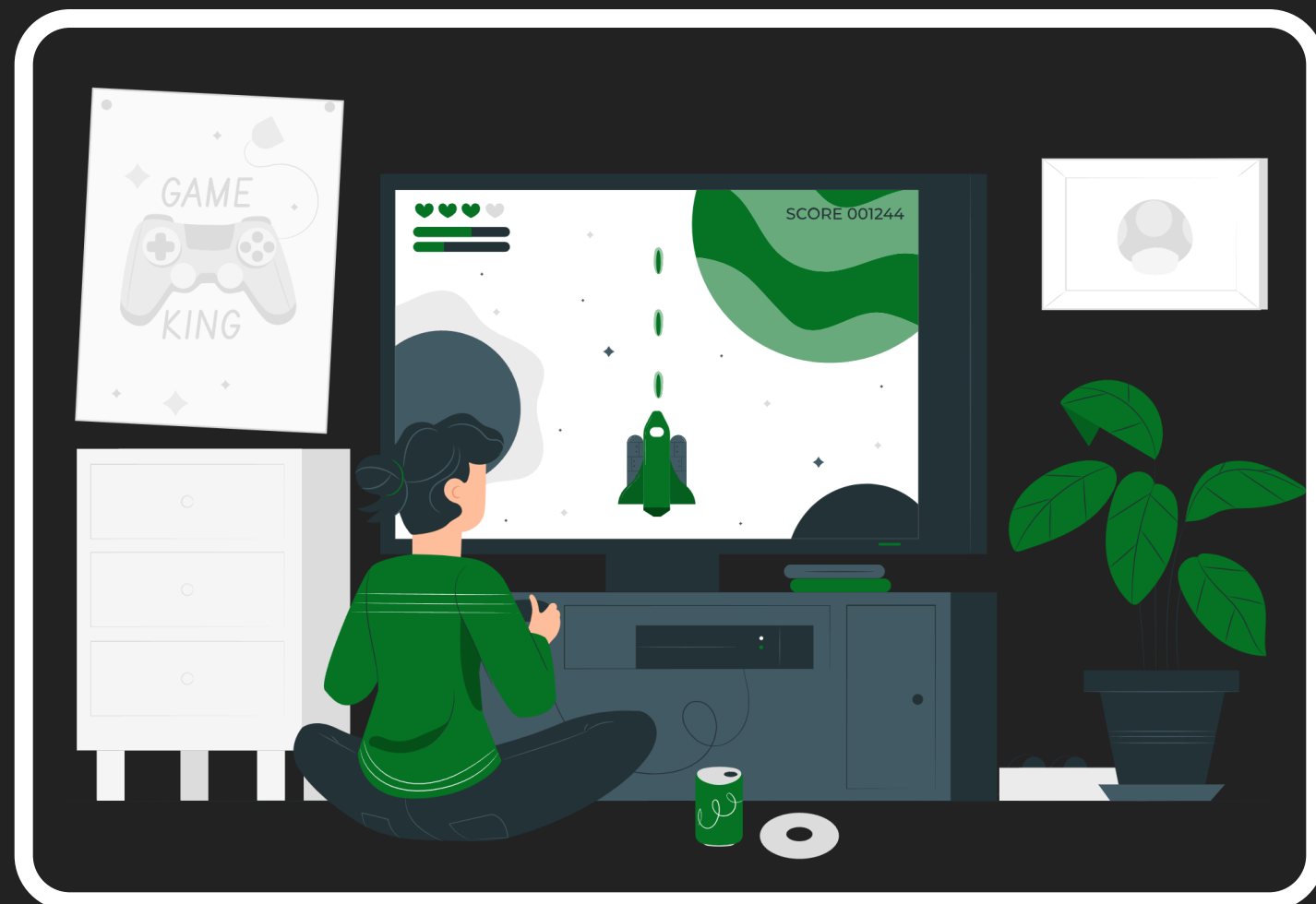
إطار عمل لمحرك ألعاب تكتيكية مبني بلغة Java، مصمم بحيث يمكن توسيعه وبناء ألعاب مختلفة فوقه. التطبيق الحالي عبارة عن لعبة ألغاز شبكية حيث يتحرك اللاعب على مربعات، يجمع العملات، يدفع الصناديق لحجب الأعداء، وعند جمع كل العملات يفتح الباب وينتقل للمستوى التالي.

● نمط Prototype يسمح بتسجيل أي نوع عدو وإنتاجه بسهولة

● نمط Composite يسمح بإضافة أي كائن للعالم دون تعديل حلقة اللعبة

● نمط Observer يسمح لأي حدث بتشغيل أي رد فعل بشكل مستقل





نمط Prototype

لماذا استخدمناه؟

يتيح نمط Prototype إنشاء نسخ من كائنات موجودة بدلاً من بنائها من جديد، مما يدعم مبدأً Open/Closed

إنشاء أعداء متعددين متشابهين



الحاجة لإنشاء أعداء متعددين متشابهين دون بناء كل منهم من الصفر، مما يوفر الوقت والجهد في التطوير

فصل بناء الكائن عن إعداده



الاستنساخ يفصل عملية بناء الكائن عن إعداده، فالقالب يحتوي على كل الخصائص مسبقاً وجاهزة للنسخ

ضمان الاتساق وسهولة التوسع

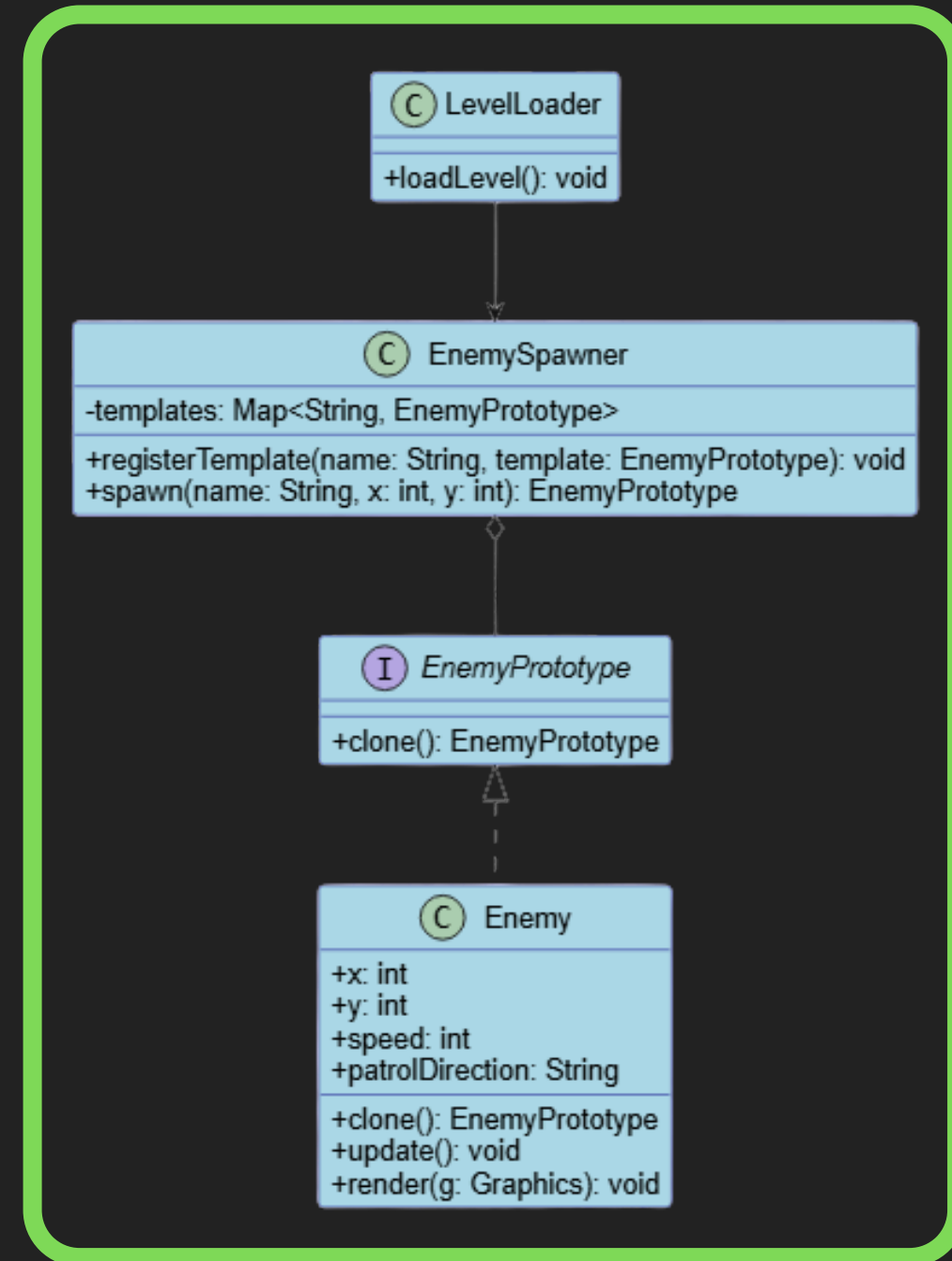


إضافة أنواع جديدة من الأعداء دون تعديل منطق الإنتاج، مع الحفاظ على اتساق الكائنات المستنسخة



نمط Prototype

شرح الكلاسات والواجهات



Enemy

النموذج الأصلي الملموس الذي
ينفذ clone() ويحتوي على جميع
الخصائص والسلوكيات.



EnemyPrototype

واجهة تعريفية تحتوي على عقد
الاستنساخ clone() الذي تنفذه
جميع الكائنات القابلة للنسخ.



LevelLoader

العميل الذي يطلب استنساخ
الأعداء من EnemySpawner دون
معرفة تفاصيل البناء.



EnemySpawner

سجل يحتفظ بقوالب الأعداء
المختلفة ويستنسخها حسب
الطلب دون إعادة البناء.



نمط Prototype

سؤال جواب

مناقشة سؤال محتمل مع الإجابة النموذجية

سؤال: لماذا لا تستخدم new Enemy () في كل مرة بدلاً من الاستنساخ؟

جواب: الاستنساخ يفصل بناء الكائن عن إعداداته، القالب يحتوي على كل الخصائص مسبقاً، وهذا يدعم مبدأ Open/Closed





نمط Composite لماذا استخدمناه؟

يوفر هذا النمط طريقة موحدة لإدارة جميع كائنات اللعبة في حلقة التحديث والرسم بكفاءة عالية

إدارة التحديث والرسم في إطار واحد



الحاجة لإدارة تحديث ورسم كل كائن في اللعبة ضمن إطار زمني واحد، مما يتطلب آلية موحدة للتنسيق

تجنب معرفة حلقة اللعبة بكل قائمة منفصلة



بدون Composite، ستحتاج حلقة اللعبة لمعرفة كل قائمة كائنات بشكل منفصل مما يزيد التعقيد والاقتران

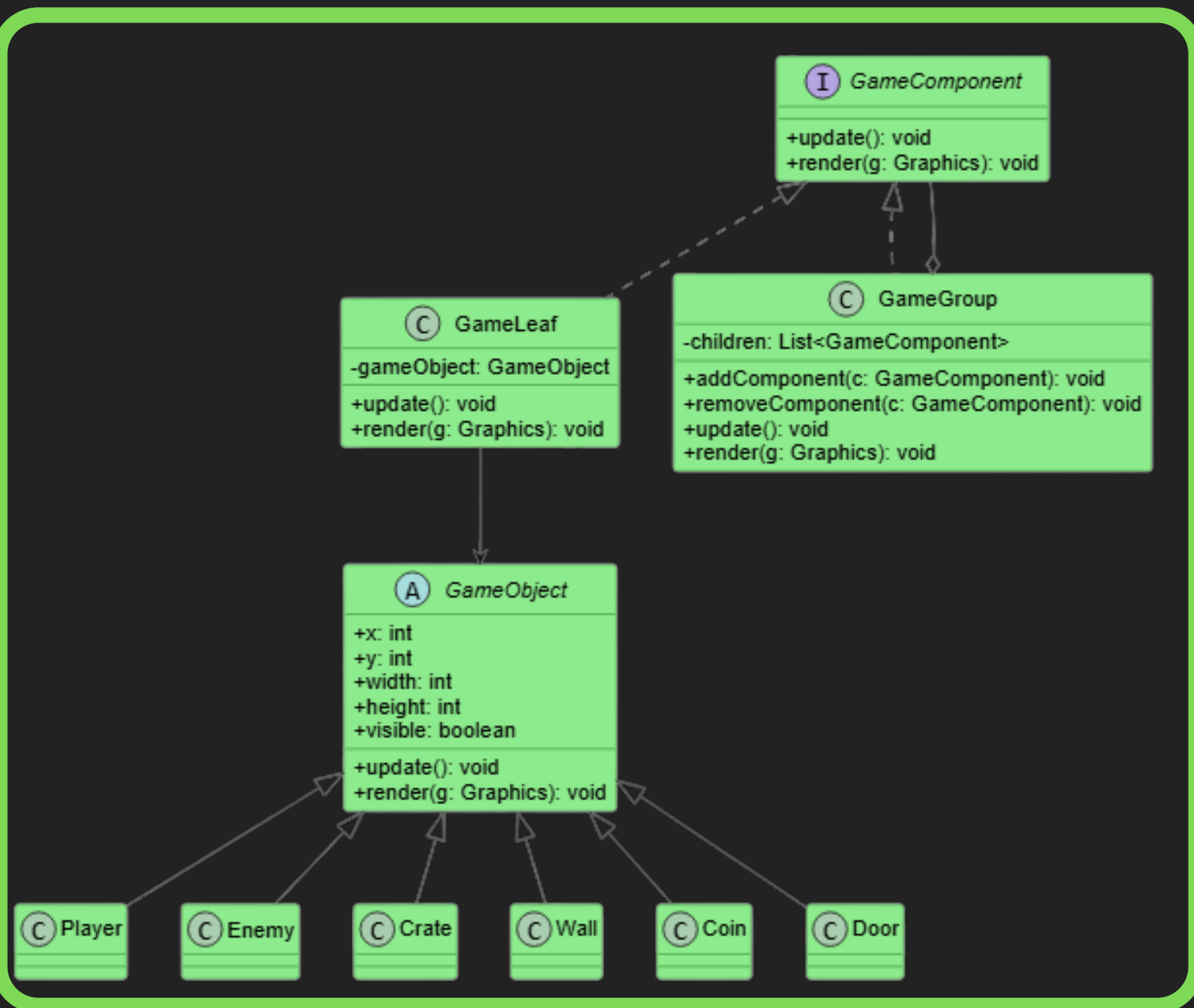
التعامل الموحد مع الكائنات والمجموعات



النمط يسمح بالتعامل مع الكائنات الفردية والمجموعات بنفس الطريقة، مما يبسط الكود ويسهل التوسع

نمط Composite

شرح الكلاسات



GameGroup

يحتوي قائمة من
GameComponent، يدير
التكرار الهرمي على جميع
الأبناء



GameObject

القاعدة المجردة لكل كائنات
اللعبة مثل Player و Enemy و
Crate



GameComponent

الواجهة الموحدة للكائنات
الفردية والمجموعات، تحدد
عقد update() و render()



GameLeaf

يغلف كائن GameObject واحد،
يفوض استدعاءات التحديث
والرسم للكائن المغلف



نمط Composite سؤال جواب

سؤال: أليس هذا مجرد قائمة؟

القائمة المسطحة تجبر المستدعي على معرفة المحتوى والتكرار يدوياً، بينما Composite يسمح بالتجميع الهرمي والتعامل الموحد

هذا يسهل التوسع ويقلل التعديل في حلقة اللعبة، مما يجعل إضافة مجموعات جديدة أمراً بسيطاً دون تغيير الكود الأساسي





نمط Observer

لماذا استخدمناه؟

يتيح Observer التواصل المرن بين مكونات اللعبة دون اقتران مباشر، مما يسهل الصيانة والتوسع

إطلاق ردود فعل متعددة عند حدث واحد

عند جمع عملة مثلاً، نحتاج تحديث العداد وتشغيل صوت وفحص فتح الباب - كل هذا من حدث واحد فقط

تجنب اقتران Player بالمكونات الأخرى

بدون Observer، سيحتاج Player معرفة CoinManager و DoorController و AudioManager مما يزيد التعقيد

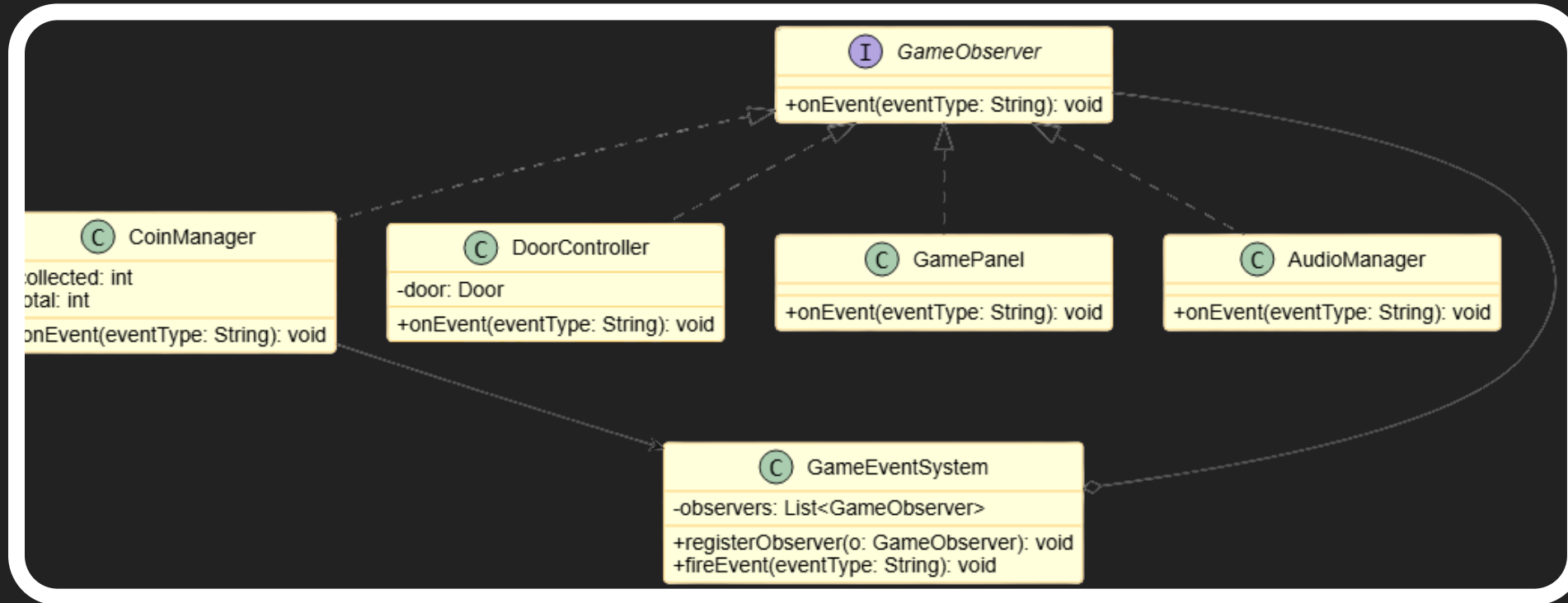
فصل مصدر الحدث عن ردود الفعل

Observer يضمن استقلالية المكونات حيث يطلق المصدر الحدث والمستمعون يستجيبون دون ربط مباشر

نمط Observer

شرح الكلاسات

الواجهات والكلاسات الرئيسية التي تشكل بنية نمط Observer في اللعبة وكيفية تفاعلها معاً



AudioManager

يشغل الصوت المناسب
عند كل حدث عملة، فوز،
أو خسارة

GameEventSystem

ناقل الأحداث يحتفظ بقائمة
المستمعين ويخطرهم عند كل
حدث

CoinManager

يعد العملات المجموعة عند
اكتمالها يطلق حدث فتح
الباب

GamePanel

يغير حالة اللعبة يعرض
شاشة الفوز أو الخسارة
عند استقبال الحدث

DoorController

يستمتع لاكتمال العملات يفتح
الباب عند استقبال الحدث

GameObserver

واجهة المستمع تفرض على كل
كلاس تنفيذ onEvent()

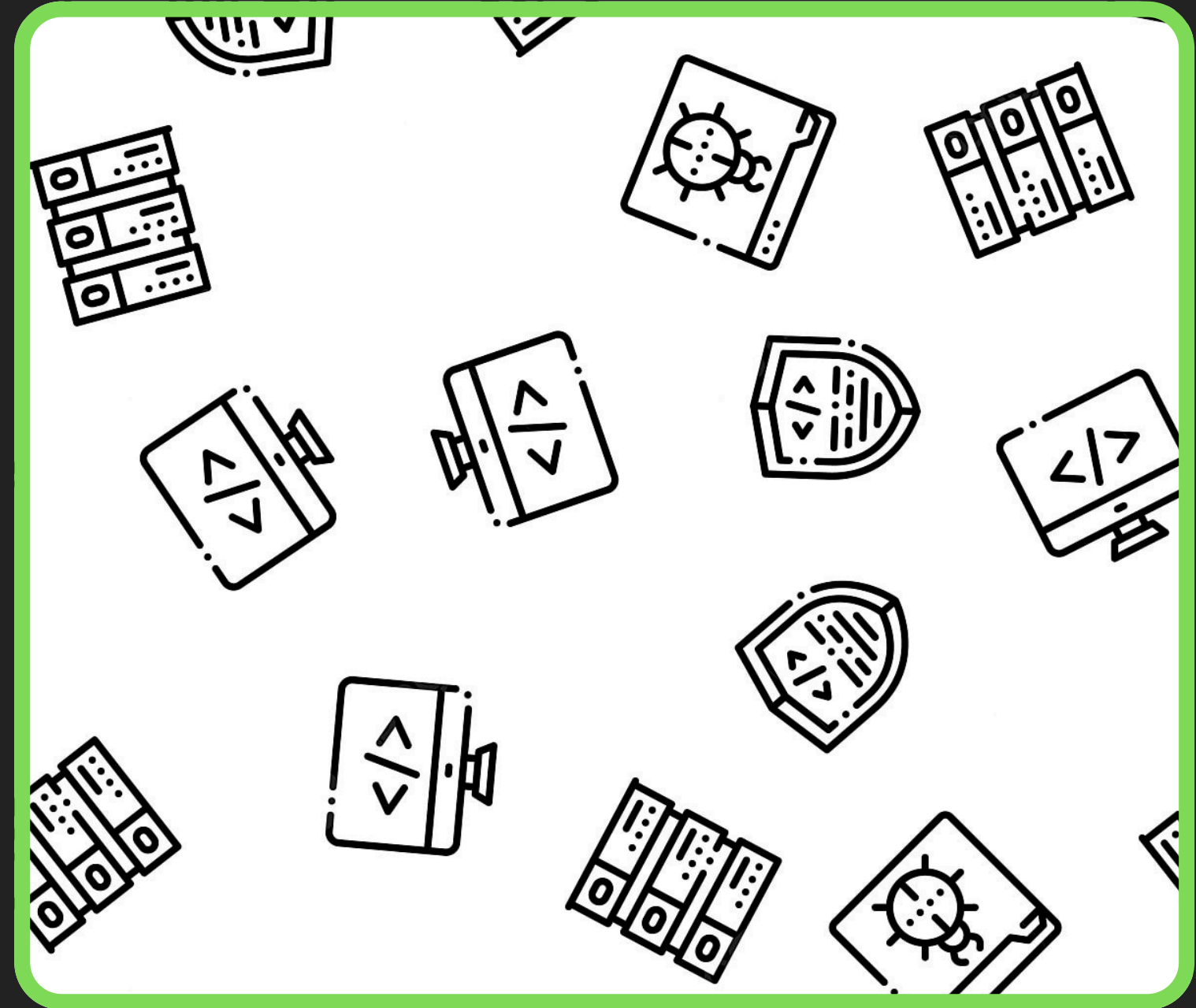
نمط Observer

سؤال جواب

لماذا لا تربط Player مباشرة بالكائنات؟

سؤال: لماذا لا تربط Player مباشرة بالكائنات الأخرى مثل CoinManager و DoorController بدلاً من استخدام نظام الأحداث؟

جواب: الربط المباشر يزيد الاقتران ويصعب الصيانة. Observer يفصل المصدر عن المتلقين، ويجعل إضافة تفاعلات جديدة سهلة بدون تعديل Player.





ملخص الأنماط الثلاثة

نظرة شاملة على كل نمط واستخدامه الأساسي في تطوير الألعاب لتحقيق كود منظم وقابل للتوسع.

● **Prototype:** إنشاء نسخ متطابقة بسهولة من كائنات معقدة

● **Composite:** تنظيم شجري للكائنات للتحديث والرسم الموحد

● **Observer:** فصل الأحداث عن ردود الفعل لزيادة المرونة





SOLID

Single Responsibility

المسؤولية الفردية كل كلاس له مهمة واحدة فقط. GameEventSystem مهمته الإخطار فقط. AudioManager مهمته تشغيل الصوت فقط. Wall مهمتها رسم نفسها فقط.

Open/Closed

مفتوح للتوسع مغلق للتعديل أنواع الأعداء الجديدة تنفذ EnemyPrototype دون تعديل EnemySpawner. المراقبون الجدد ينفذون GameObserver دون تعديل GameEventSystem.

Liskov Substitution

مبدأ استبدال Liskov أي فئة فرعية من GameObject يمكن أن تحل محل GameObject دون كسر النظام. أي منفذ لـ GameComponent يتصرف بشكل صحيح عند استدعائه عبر الواجهة.

Interface Segregation

فصل الواجهات GameComponent تحتوي فقط على update() و GameObserver. render() تحتوي فقط على onEvent(). لا يُجبر أي كلاس على تنفيذ دوال لا يحتاجها.

Dependency Inversion

عكس التبعية EnemySpawner يعتمد على واجهة EnemyPrototype وليس على كلاس Enemy مباشرة. GameEventSystem يعتمد على واجهة GameObserver وليس على CoinManager مباشرة.





شكرا لكم

